

AIConnex — Backend Architecture Reference

Complete MLOps Pipeline Documentation for Frontend Engineering Team

Document Version: v1.0 | **Last Updated:** 2026-07-22 | **Status:** Active Development

Repository: github.com/AJ6002/aiconnex_demo

Diagram Direction Convention: All flow diagrams are rendered **Left → Right** (LR) for documentation consistency.

Table of Contents

1. [System Overview](#)
2. [High-Level Architecture](#)
3. [Data Flow — End-to-End](#)
4. [Node-by-Node Deep Dive](#)
5. [DAG Routing Architecture](#)
6. [Validation Gate Architecture \(VG_1 & VG_2\)](#)
7. [Sequence Diagrams](#)
8. [Universal Relational Compiler Module](#)
9. [External Package Registry](#)
10. [Backend Constants & Configuration](#)
11. [Artifact Storage Schema](#)
12. [Current Testing Scope](#)

1. System Overview

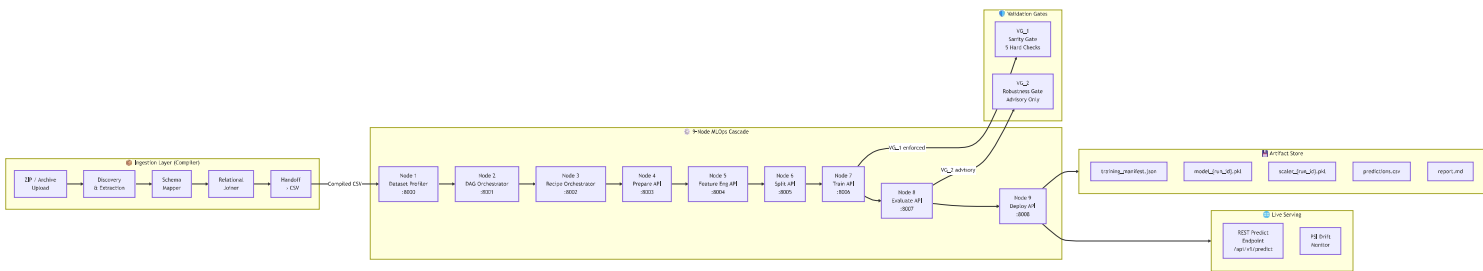
AIConnex is a **9-node, microservice-based, automated Industrial MLOps platform**. Each node is an independent REST API (FastAPI + Uvicorn). They are orchestrated sequentially by a central pipeline

runner (run_pipeline.py) which forwards data, metadata, and model artifacts via shared filesystem paths and JSON manifests.

The system operates on two parallel tracks:

Track	Scope	Purpose
Universal Relational Compiler	Pre-Pipeline	Ingest raw multi-file SCADA/industrial archives → clean unified CSV
9-Node MLOps Cascade	Core Pipeline	Dataset Profiling → DAG Routing → Recipe → Prepare → FeatEng → Split → Train → Evaluate → Deploy

2. High-Level Architecture



3. Data Flow — End-to-End



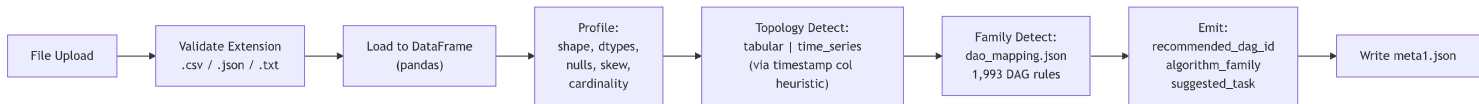
4. Node-by-Node Deep Dive

Node 1 — Dataset Profiler (port: 8000)

Property	Value
File	aic/1_dataset_profiler/main.py

Property	Value
Framework	FastAPI + Uvicorn
Key Endpoints	GET /api/v1/health · POST /api/v1/profile
Input	Uploaded .csv / .json / .txt file + optional target_column
Output	Profile JSON → meta/meta1.json
Core Modules	profiler.py (statistical profiling) · detector.py (family/topology detection)

Internal Processing Pipeline:



Key Output Fields:

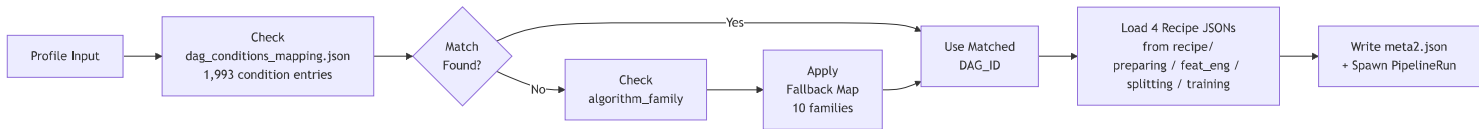
```
{
  "run_id": "run_5cffcec1",
  "recommended_dag_id": "DAG_906",
  "algorithm_family": "Time-Series",
  "suggested_task": "Time Series Analysis",
  "topology": "time_series",
  "target_column": "pnb950657_tt02",
  "shape": [1798, 27],
  "profile": { ... }
}
```

Node 2 — DAG Orchestrator (port: 8001)

Property	Value
File	aic/2_dag/orchestrator.py + aic/2_dag/main.py
Key Endpoints	POST /api/v1/run · GET /api/v1/run/{run_id}/status
Input	Profile payload from Node 1
Output	PipelineRun object · meta2.json (4 stage recipes merged)

Property	Value
Fallback Map	10-family DAG fallback table (Classification→ DAG_001 , Regression→ DAG_283 , Time-Series→ DAG_1059 , etc.)

DAG Selection Logic:



10-Family Fallback Registry:

Classification	→ DAG_001	(F1, DAG_001–DAG_282)
Regression	→ DAG_283	(F2, DAG_283–DAG_572)
Anomaly Detection	→ DAG_573	(F3, DAG_573–DAG_819)
Clustering	→ DAG_820	(F4, DAG_820–DAG_1058)
Time-Series	→ DAG_1059	(F5, DAG_1059–DAG_1315)
Digital Twin	→ DAG_1316	(F6, DAG_1316–DAG_1450)
Reinforcement Lrn.	→ DAG_1451	(F7, DAG_1451–DAG_1571)
Recommendation	→ DAG_1572	(F8, DAG_1572–DAG_1704)
NLP / Text-Class.	→ DAG_1705	(F9, DAG_1705–DAG_1836)
Computer Vision	→ DAG_1837	(F10, DAG_1837–DAG_1993)

Node 3 — Recipe Orchestrator (port: 8002)

Property	Value
File	aic/3_recipe_orchestrator/main.py
Key Endpoints	POST /api/v1/recipe • GET /api/v1/recipe/status
Master Catalog	aic/3_recipe_orchestrator/recipe/ — 7,972 JSON files across 4 stage subdirectories
Output	training_manifest_{run_id}.json — unified recipe for all pipeline stages

Recipe Directory Layout:

```
aic/3_recipe_orchestrator/recipe/
├─ preparing/           (1,993 JSONs: DAG_001.json ... DAG_1993.json)
├─ feature_engineering/ (1,993 JSONs)
├─ splitting/          (1,993 JSONs)
└─ training/           (1,993 JSONs)
```

Node 4 — Prepare API (port: 8003)

Property	Value
File	aic/4_prepare/main.py
Key Endpoints	POST /api/v1/prepare
Input	raw_file_path , recipe , run_id , target_column , manifest_path
Output	prepared_{run_id}.csv → workspace_data/{run_id}/
External Libs	pandas , numpy , sklearn.preprocessing (StandardScaler, MinMaxScaler, LabelEncoder)

Transformations Applied:

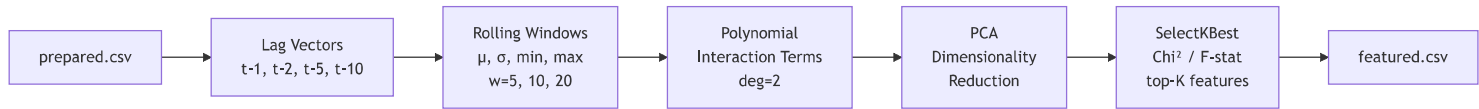
- Numeric column imputation (mean/median/constant)
- Categorical encoding (Label or One-Hot)
- Feature scaling (Standard / MinMax / Robust)
- Chronological sort enforcement for `time_series` topology
- Outlier winsorization (optional)

Node 5 — Feature Engineering API (port: 8004)

Property	Value
File	aic/5_feature_engineering/main.py
Key Endpoints	POST /api/v1/feature_engineer • POST /api/v1/chart_plot

Property	Value
Output	featured_{run_id}.csv
External Libs	pandas , numpy , sklearn.decomposition (PCA) , sklearn.feature_selection (SelectKBest) , aiconnex_ml

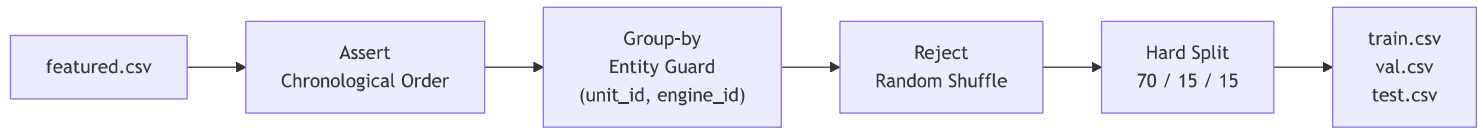
Feature Transformations:



Node 6 — Split API (port: 8005)

Property	Value
File	aic/6_split/main.py
Key Endpoints	POST /api/v1/split
Default Ratios	70% train · 15% val · 15% test
Output	train.csv , val.csv , test.csv → workspace_data/{run_id}/splits/

Zero-Leakage Guarantee:



Node 7 — Train API (port: 8006)

Property	Value
File	aic/7_train/main.py
Key Endpoints	POST /api/v1/train (202 Accepted, async) · GET /api/v1/train/status/{job_id}

Property	Value
Input	train_path , val_path , target_column , recipe , run_id , manifest_path
Output	model_{run_id}.pkl · scaler_{run_id}.pkl
HPO	25-iteration RandomizedSearchCV
Async	Returns 202 Accepted with job_id ; poll /status/{job_id} for completion

13 Supported Algorithm Classes:

1. LogisticRegression (Classification)
2. LinearRegression (Regression)
3. HuberRegressor (Robust Regression)
4. Ridge (L2 Regularization)
5. Lasso (L1 Sparsity)
6. ElasticNet (L1+L2 Joint)
7. GradientBoosting[R/C] (GBDT Ensemble)
8. AdaBoost[R/C] (Adaptive Boosting)
9. ExtraTrees[R/C] (Extremely Randomized Trees)
10. RandomForest[R/C] (Bagging Ensemble)
11. DecisionTree[R/C] (Single CART Tree)
12. SVR / SVC (Support Vector Machine)
13. KNeighbors[R/C] (K-Nearest Neighbors)
 - + IsolationForest (Anomaly Detection)
 - + KMeans (Clustering)

Node 8 — Evaluate API (port: 8007)

Property	Value
File	aic/8_evaluate/main.py
Key Endpoints	POST /api/v1/evaluate
VG_2 Mode	Advisory — gate score logged, never blocks deployment
Input	model_path , test_path , target_column , metrics , manifest_path
Output	predictions_{run_id}.csv · report_{run_id}.md · VG_2 gate report

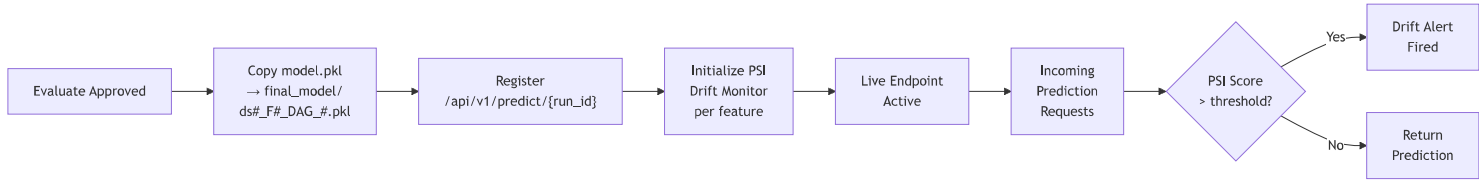
Metrics by Topology:

Topology	Metrics Computed
regression / time_series	R², MAE, RMSE, MAPE, NRMSE
classification	Accuracy, F1 (weighted), Precision, Recall
anomaly_detection	Contamination, Anomaly Score Distribution
clustering	Silhouette Score, Inertia

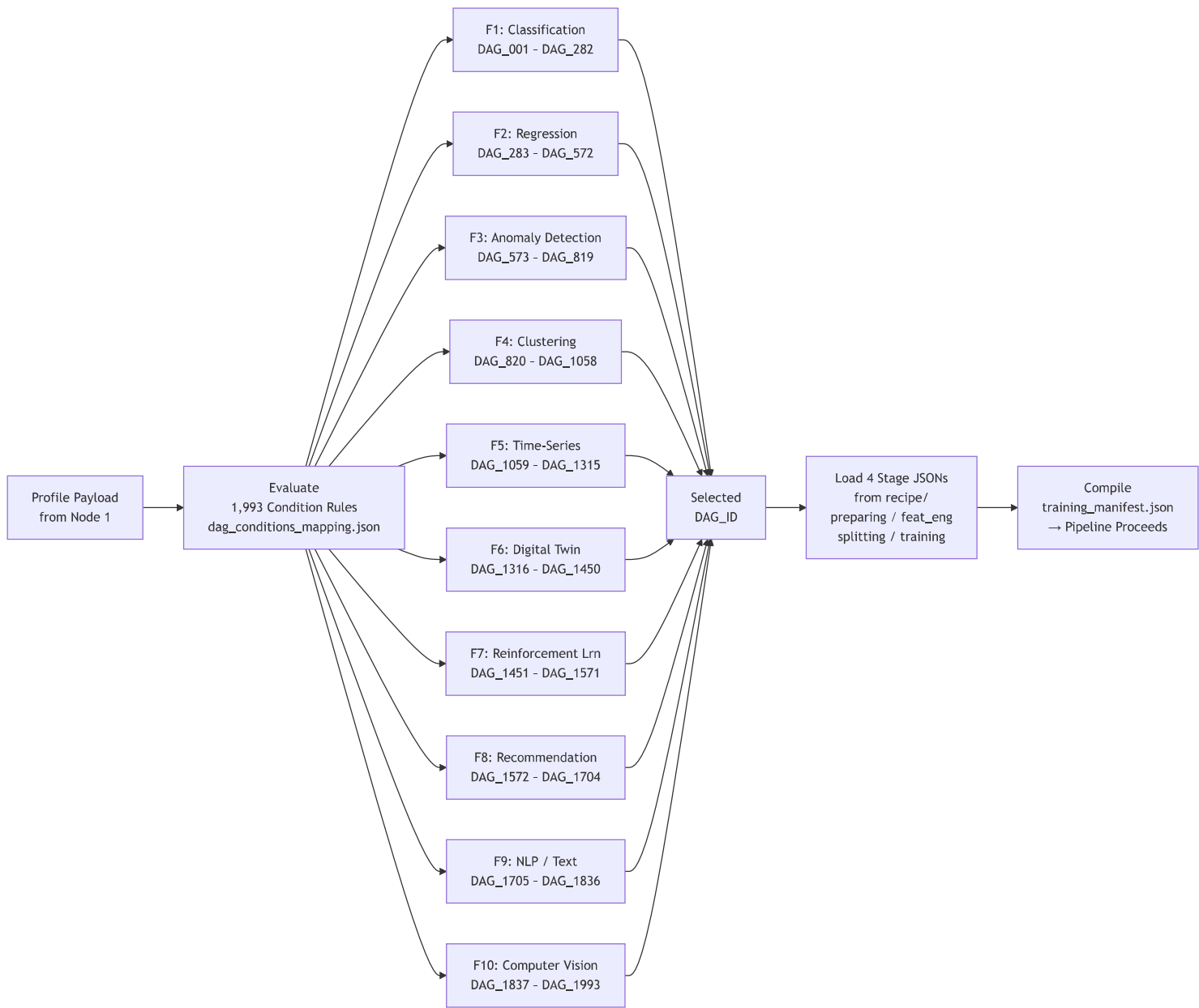
Node 9 — Deploy API (port: 8008)

Property	Value
File	aic/9_deploy_monitor/main.py
Key Endpoints	POST /api/v1/deploy · GET /api/v1/health
Model Naming	{ds_number}_{family_id}_{dag_id}.pkl (e.g. ds1_F5_DAG_906.pkl)
Model Registry	aic/final_model/ directory
External Libs	pandas , shutil , algorithm_families_complete.xlsx

Deployment + PSI Serving:



5. DAG Routing Architecture



Master DAG Registry Summary:

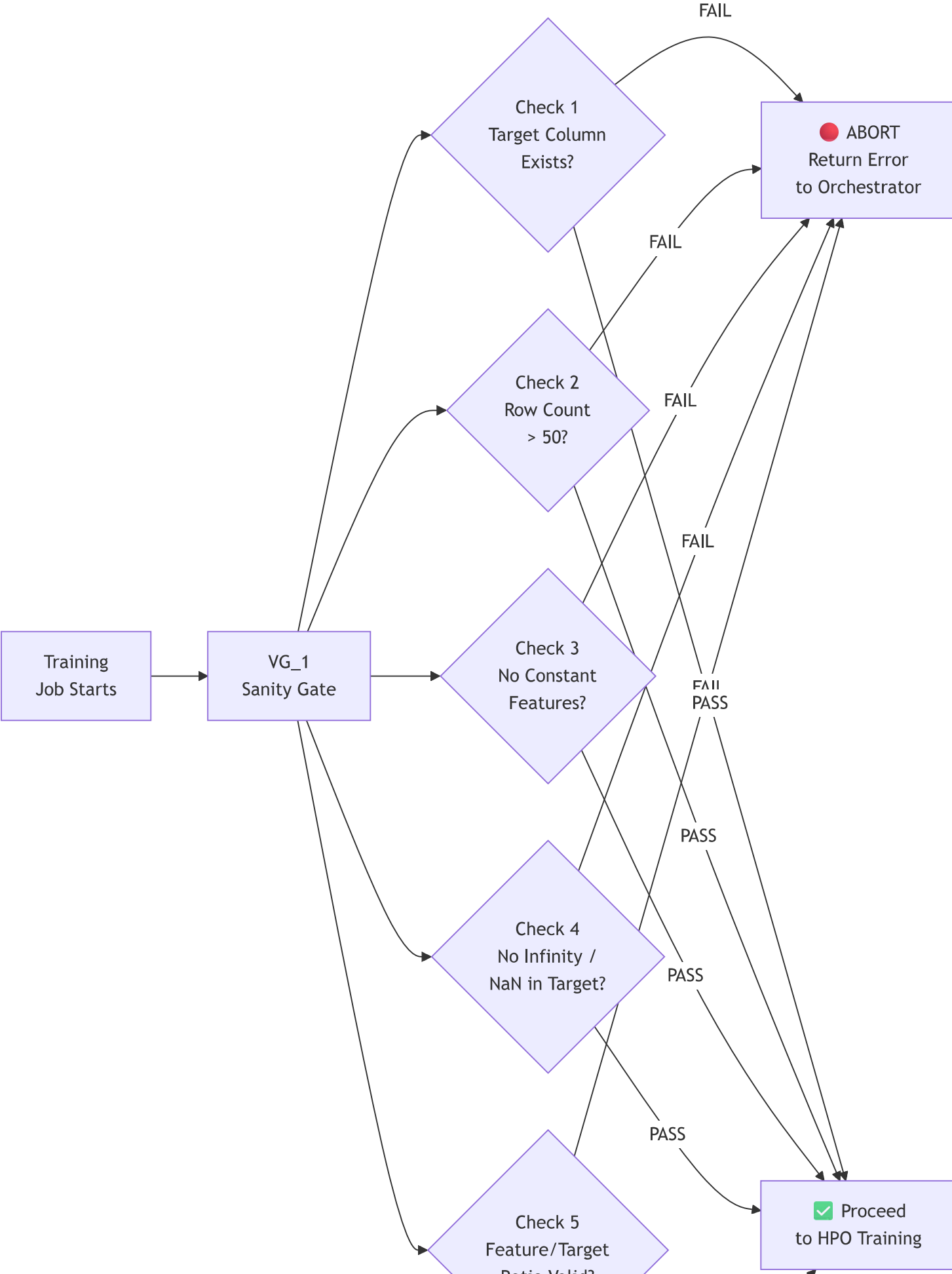
Family	ML Domain	DAG Range	Total DAGs	Fallback DAG
F1	Classification	DAG_001 — DAG_282	282	DAG_001
F2	Regression	DAG_283 — DAG_572	290	DAG_283
F3	Anomaly Detection	DAG_573 — DAG_819	247	DAG_573
F4	Clustering	DAG_820 — DAG_1058	239	DAG_820
F5	Time-Series	DAG_1059 — DAG_1315	257	DAG_1059

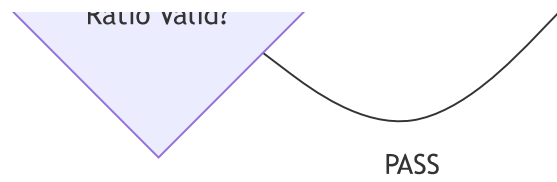
Family	ML Domain	DAG Range	Total DAGs	Fallback DAG
F6	Digital Twin	DAG_1316 — DAG_1450	135	DAG_1316
F7	Reinforcement Learning	DAG_1451 — DAG_1571	121	DAG_1451
F8	Recommendation	DAG_1572 — DAG_1704	133	DAG_1572
F9	NLP / Text-Classification	DAG_1705 — DAG_1836	132	DAG_1705
F10	Computer Vision	DAG_1837 — DAG_1993	157	DAG_1837
	TOTAL		1,993	

6. Validation Gate Architecture (VG_1 & VG_2)

VG_1 — Hard Sanity Gate (Node 7, Blocking)

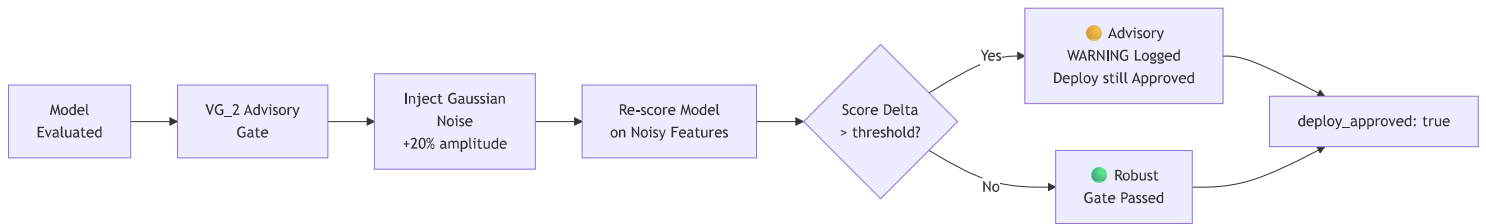
VG_1 runs **before** model training begins. All 5 checks must pass. Failure aborts the training job and returns a clear error to the orchestrator.





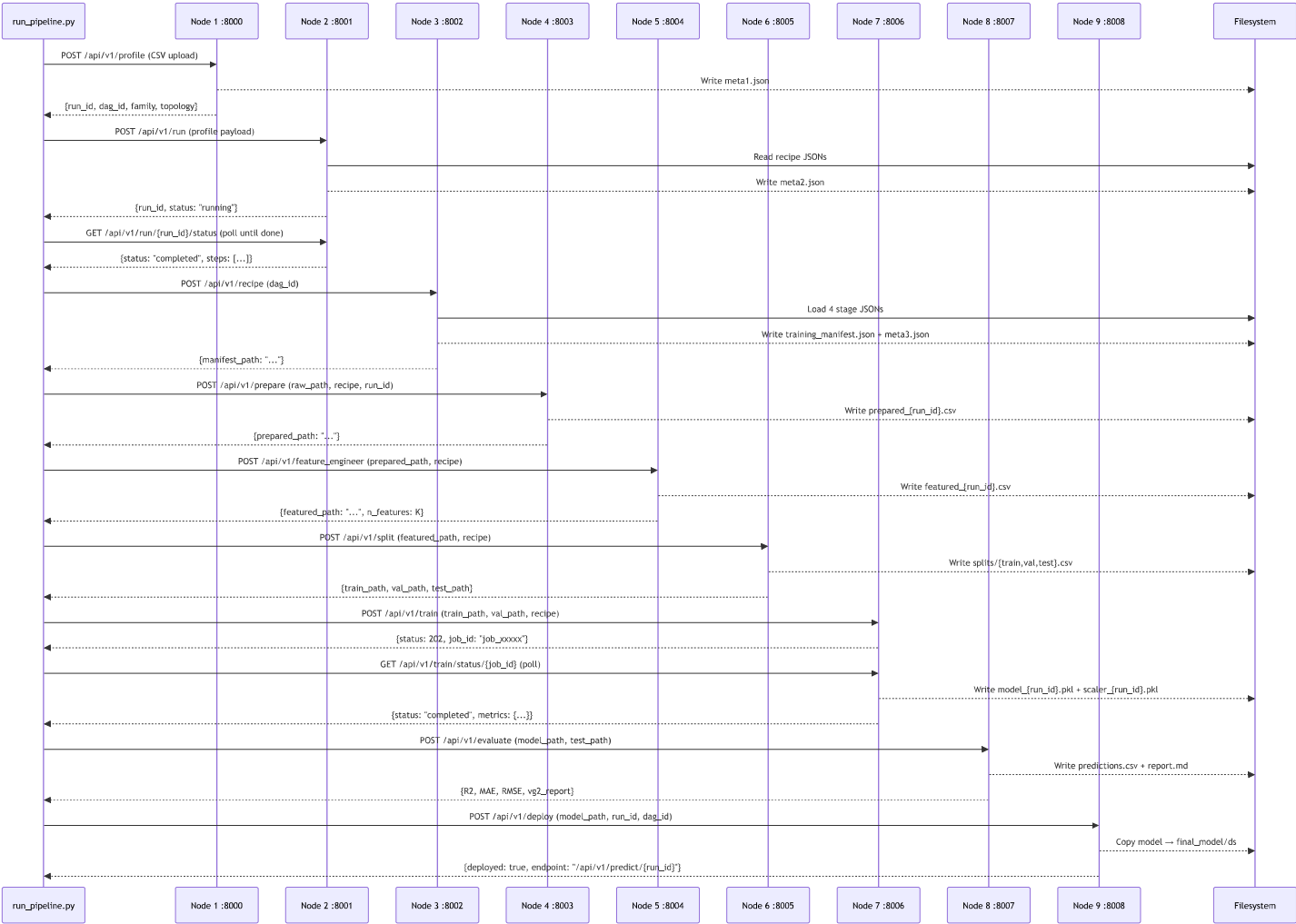
VG_2 — Robustness Advisory Gate (Node 8, Non-Blocking)

VG_2 runs **after** model evaluation. It is **advisory only** — `deploy_approved` is always `true`. The gate scores model robustness under synthetic noise injection.

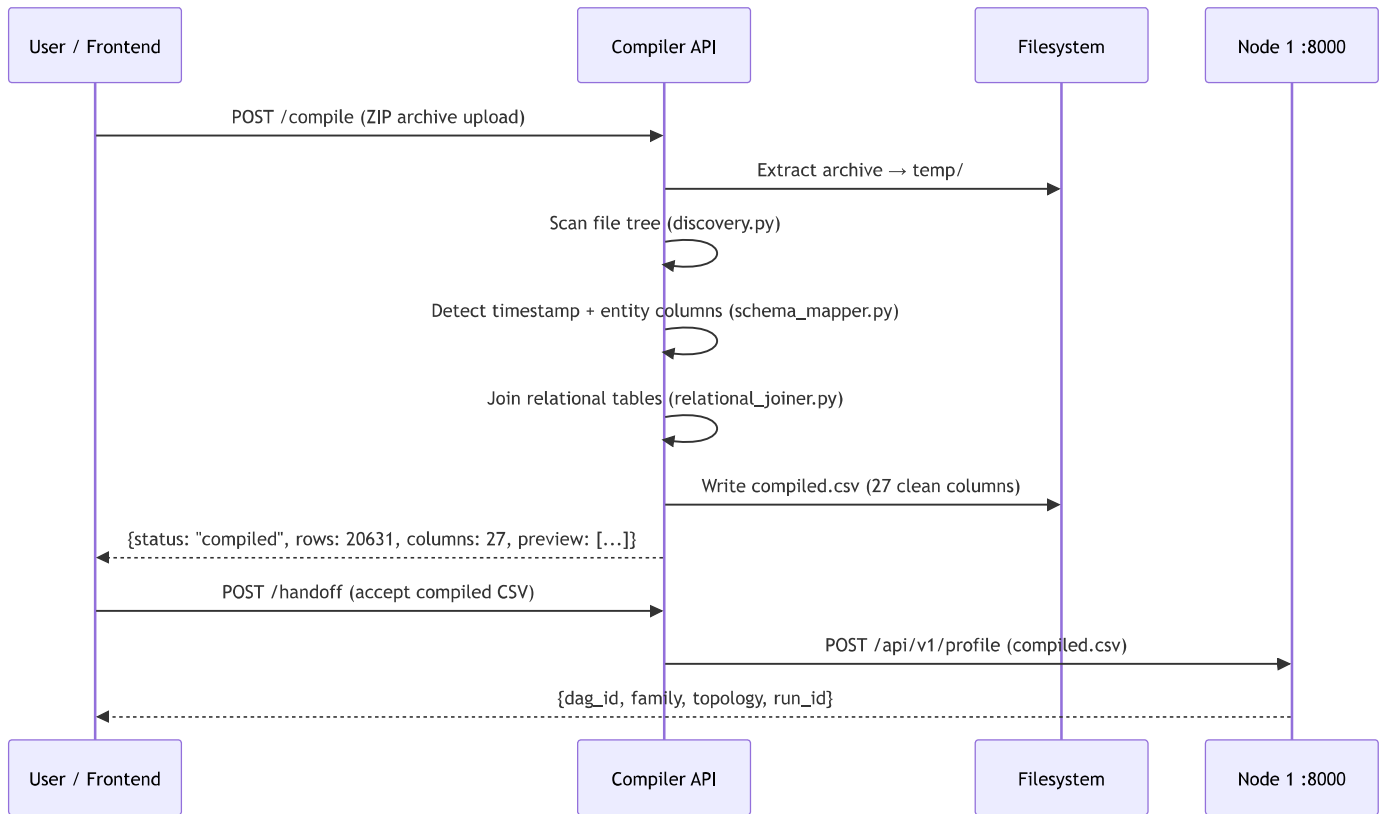


7. Sequence Diagrams

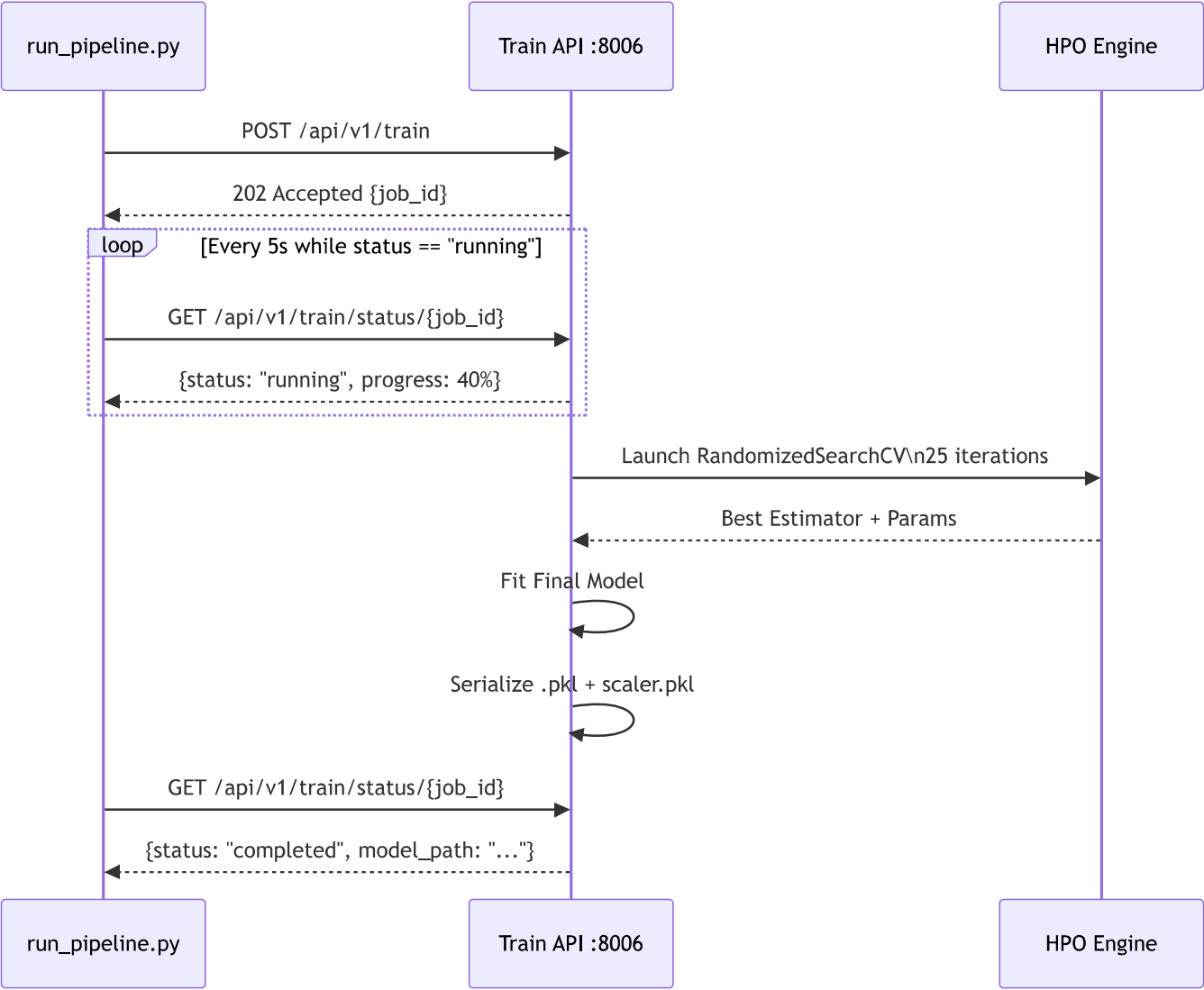
7.1 Full Pipeline Run — Happy Path



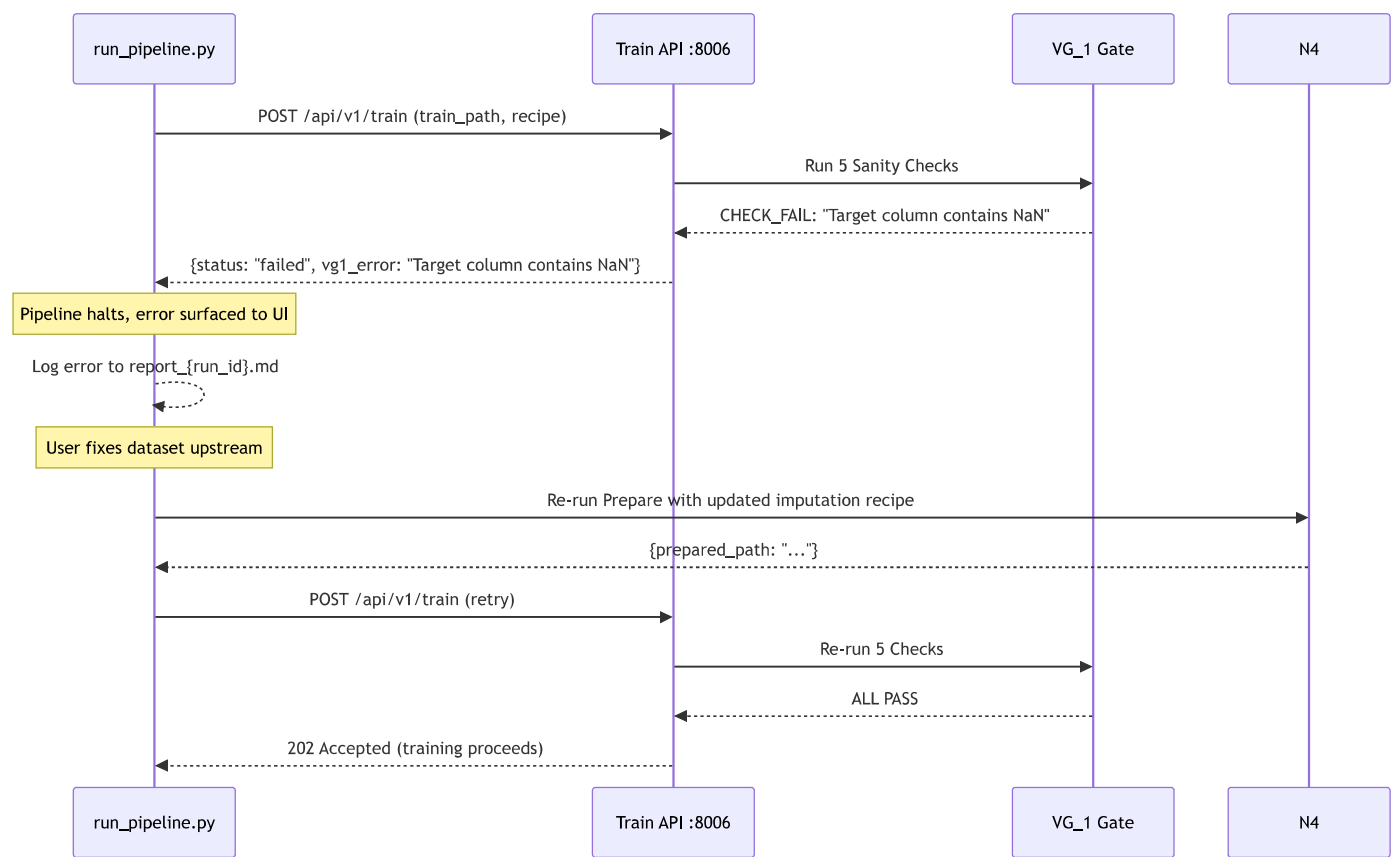
7.2 Compiler → Pipeline Handoff Sequence



7.3 Async Training Job Polling Sequence



7.4 VG_1 Failure & Recovery Sequence

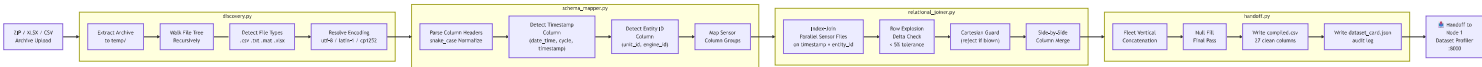


8. Universal Relational Compiler Module (Isolated)

Important

The Compiler is a **separate module** from the 9-Node MLOps pipeline. It is implemented but **backend completion is still in progress**. Frontend should build the Compiler UI section as an isolated workspace tab (/compiler).

Directory: aiconnex_zip_compiler/



Compiler Module Files:

File	Role	Status
cli.py	CLI entrypoint	✓ Active
compiler.py	Main 4-layer orchestration	✓ Active
discovery.py	Archive extraction & file tree scan	✓ Active
schema_mapper.py	Column normalization & timestamp detection	✓ Active
relational_joiner.py	Index-join & row explosion guard	✓ Active
handoff.py	Fleet concatenation & dataset_card.json	✓ Active
snapshot_aggregator.py	Multi-run snapshot generation	🔄 In Progress
AI Data Agent	NL query interface over compiled tables	📅 Planned
Rejection API	REST endpoint for rejection responses	📅 Planned

9. External Package Registry

Core Backend Framework

Package	Version (approx)	Used In	Purpose
fastapi	0.110+	All 9 nodes	REST API framework
uvicorn	0.29+	All 9 nodes	ASGI server
pydantic	2.x	All 9 nodes	Request/Response validation

Data Science & Processing

Package	Used In	Purpose
pandas	N1, N4, N5, N6, N8, N9	DataFrame operations, CSV I/O
numpy	N4, N5, N7, N8	Numerical computation, array math
scipy	N5 (via aiconnex_ml)	Statistical tests, FFT

Package	Used In	Purpose
openpyxl	N9, Compiler	Read .xlsx algorithm families registry

Machine Learning (sklearn)

Package / Class	Used In	Purpose
sklearn.linear_model	N7	LogisticRegression, LinearRegression, Ridge, Lasso, ElasticNet, HuberRegressor
sklearn.ensemble	N7	RandomForest, GradientBoosting, AdaBoost, ExtraTrees, IsolationForest
sklearn.tree	N7	DecisionTree[R/C]
sklearn.neighbors	N7	KNeighbors[R/C]
sklearn.svm	N7	SVR, SVC
sklearn.cluster	N7	KMeans
sklearn.preprocessing	N4, N7	StandardScaler, MinMaxScaler, LabelEncoder
sklearn.decomposition	N5	PCA
sklearn.feature_selection	N5	SelectKBest
sklearn.metrics	N8	r2_score, MAE, RMSE, accuracy, f1, silhouette
sklearn.model_selection	N7	RandomizedSearchCV (HPO, 25 iters)

Internal Package: aiconnex_ml

Module	Used In	Purpose
aiconnex_ml.regression.evaluation	N8	Custom regression metrics (NRMSE, RUL)
aiconnex_ml.anomaly.evaluation	N8	Anomaly score distributions
aiconnex_ml.time_series.*	N5, N7	Lag generation, spectral FFT features

Infrastructure

Package	Used In	Purpose
requests	run_pipeline.py	Inter-node HTTP calls
subprocess	start_all.py	Launch microservice processes
shutil	N9, Compiler	File copy operations
pickle	N7, N8	Model serialization
hashlib	N1	run_id generation
uuid	N2, N7	Job ID generation

10. Backend Constants & Configuration

Service Registry (Canonical Port Map)

```
# Source: aic/start_all.py & aic/run_pipeline.py

SERVICES = {
    "Dataset Profiler":      8000,    # Node 1
    "DAG Orchestrator":     8001,    # Node 2
    "Recipe Orchestrator":  8002,    # Node 3
    "Prepare API":          8003,    # Node 4
    "Feature Engineering API": 8004,  # Node 5
    "Split API":            8005,    # Node 6
    "Train API":            8006,    # Node 7 (Async, 202 Accepted)
    "Evaluate API":         8007,    # Node 8
    "Deploy API":           8008,    # Node 9
}
```

Path Constants (Dynamic Root Resolution)

```
# Common across all nodes (Windows-safe, machine-agnostic)
AIC_ROOT          = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
WORKSPACE_ROOT    = os.path.join(AIC_ROOT, "workspace_data")
FINAL_MODEL_DIR   = os.path.join(AIC_ROOT, "final_model")
ALGO_FAMILIES_XLS = os.path.join(AIC_ROOT, "algorithm_families_complete.xlsx")
```

Data Split Ratios

```
TRAIN_RATIO = 0.70 # 70% training
VAL_RATIO   = 0.15 # 15% validation
TEST_RATIO  = 0.15 # 15% holdout test
```

HPO Configuration

```
HPO_N_ITER      = 25      # RandomizedSearchCV iterations
HPO_CV_FOLDS    = 3       # Cross-validation folds
HPO_SCORING_REG = "r2"    # Regression scoring
HPO_SCORING_CLF = "f1_weighted" # Classification scoring
HPO_N_JOBS      = -1      # Use all available CPU cores
```

VG_1 Sanity Checks

```
VG1_MIN_ROWS      = 50      # Minimum dataset rows
VG1_MAX_CONSTANT_RATIO = 0.99 # Column constant variance threshold
VG1_TARGET_NAN_ALLOWED = False # Target NaN tolerance
```

Compiler Constants

```
COMPILER_ROW_EXPLOSION_THRESHOLD = 0.05 # 5% max row explosion on join
COMPILER_SUPPORTED_FORMATS = [".csv", ".txt", ".xlsx", ".mat", ".zip"]
COMPILER_DEFAULT_ENCODING = "utf-8"
COMPILER_FALLBACK_ENCODING = "latin-1"
```

Model Naming Convention

```
# final_model/{ds_number}_{family_id}_{dag_id}.pkl
# Example: ds1_F5_DAG_906.pkl
#   ds1   = dataset number
#   F5     = algorithm family (Time-Series)
#   DAG_906 = selected DAG
```

11. Artifact Storage Schema

All run artifacts are stored in `workspace_data/{run_id}/` :

```
workspace_data/
└─ {run_id}/
    ├── training_manifest_{run_id}.json    ← Unified pipeline configuration
    ├── prepared_{run_id}.csv             ← After Node 4
    ├── featured_{run_id}.csv             ← After Node 5
    ├── splits/
    │   ├── train.csv                    ← 70% chronological split
    │   ├── val.csv                     ← 15% validation
    │   └─ test.csv                     ← 15% holdout
    ├── model_{run_id}.pkl                ← Trained model (sklearn estimator)
    ├── scaler_{run_id}.pkl              ← Feature scaler (if applied)
    ├── predictions_{run_id}.csv          ← Test-set predictions + actuals
    └─ report_{run_id}.md                ← Human-readable evaluation report
```

```
aic/final_model/
└─ ds{N}_F{N}_DAG_{N}.pkl              ← Deployed model copy
```

```
aic/2_dag/meta/
└─ meta2.json                          ← Active DAG + 4 stage recipes
```

```
aic/3_recipe_orchestrator/meta/
└─ meta3.json                          ← Compiled unified manifest
```

12. Current Testing Scope & Active Work



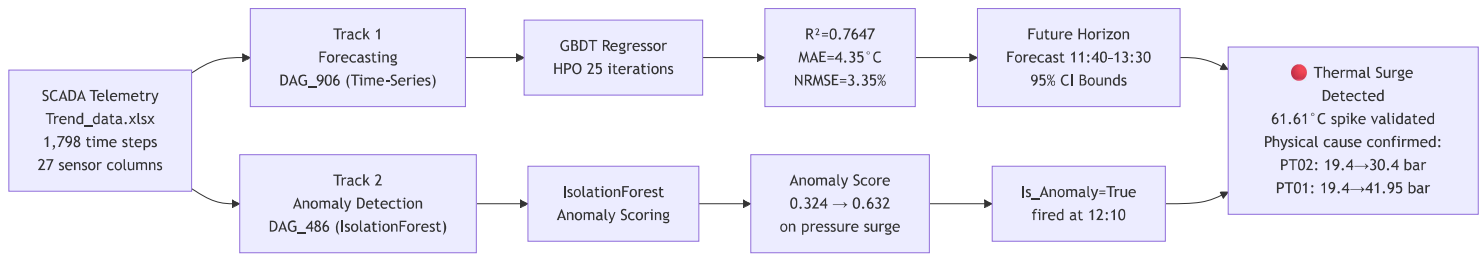
As of **2026-07-22**, the team is actively in the **Dataset Testing Phase**. This section documents what is being tested, what has been verified, and what is pending.

✓ Completed & Verified

Dataset	Topology	DAG Routed	Result
SCADA Trend Telemetry (Trend_data.xlsx)	time_series	DAG_906 (F5)	$R^2=0.7647$, $MAE=4.35^{\circ}\text{C}$, $NRMSE=3.35\%$
Medical Insurance Premiums (insurance.csv)	tabular	F2 Regression	Regression baseline verified
Manufacturing Process (manufacturing.csv)	tabular	F3 Anomaly Detection	Anomaly detection pipeline verified
House Prices (house_prices_train.csv)	tabular	F2 Regression	Advanced regression verified

🔄 Active Testing: SCADA Dual-Track Analysis

The current priority test is demonstrating **Forecasting + Anomaly Detection dual-track** on industrial SCADA telemetry:



📌 Next Testing Targets

Dataset	Planned DAG Family	Priority
IGBT Degradation (igbt_aging.csv)	F5 Time-Series / RUL	● High

Dataset	Planned DAG Family	Priority
Vibration Spectral Data	F3 Anomaly Detection	● Medium
Multi-asset Fleet CMAPSS	F5 Time-Series (Batch)	● Medium
Solar Power Generation	F2 Regression	● Low

Backend Features Pending Implementation

Feature	Node / Module	Status
AI Data Agent (NL Query over compiled tables)	Compiler Module	 Planned
Compiler REST API (expose as endpoint)	Compiler Module	 Planned
PSI Drift real-time alert webhook	Node 9	 Planned
Automated Retrain Trigger (on drift)	Node 9	 Planned
Multi-model ensemble voting	Node 7	 Planned
Explainability (SHAP values)	Node 8	 Planned
LLM-based report generation	Node 8	 Planned

Document maintained by the AIConnex backend engineering team.

Frontend team: Reference [aiconnex_ui_backend_feature_map.md](#) for the 1-to-1 backend-to-UI component mapping.